Perceptrón Multicapa para clasificación de texto

20 Newsgroups con Keras/TensorFlow — Entrega 1

Nicolás Garcés (218066) Juan Pablo Ante (2140132)

Escuela de Ingeniería de Sistemas y Computación
Universidad del Valle

2026-2

Problema y objetivos

Problema. Clasificar mensajes de Usenet en 20 categorías temáticas a partir de su texto.

Por qué es difícil.

- Texto disperso, ruidoso y de longitud variable.
- Clases temáticamente solapadas (computación, religión, política).
- Metadatos con fuga de información si no se eliminan.

Objetivo

Entrenar y evaluar una **MLP** en Keras/TensorFlow, ajustar sus hiperparámetros y analizar críticamente sus errores.

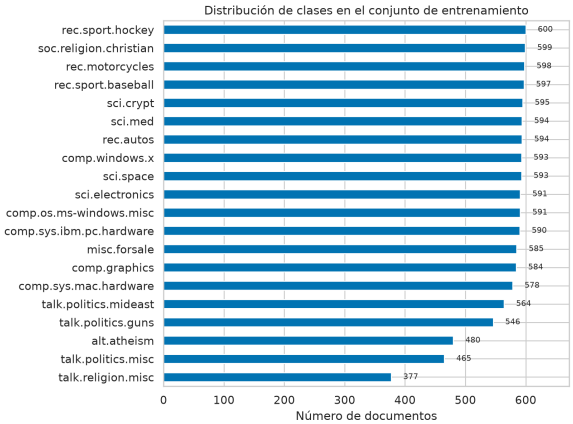
Alcance

Clasificación **multiclase** (una etiqueta por documento), tal como están anotados los datos.

Conjunto de datos: 20 Newsgroups

Documentos de entrenamiento	11,314
Documentos de prueba	7,532
Clases	20
Palabras por documento (media)	185.8
Palabras por documento (mediana)	83
Desbalance (mayor/menor)	1.59×

Partición oficial de scikit-learn. Se eliminan **cabeceras, pies y citas** para evitar que el nombre del grupo revele la etiqueta.



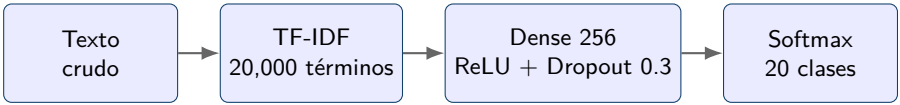
Preprocesamiento y protocolo

1. **Texto crudo**: minúsculas, sin puntuación, tokenización por espacios.
2. **TF-IDF** con vocabulario de 20,000 términos, adaptado solo con entrenamiento.
3. **Partición**: 90 % train / 10 % validación (estratificada); prueba oficial intacta hasta el final.
4. **Normalización** de cada vector TF-IDF a norma unitaria.

Por qué TF-IDF

Señal densa y discriminativa, entrenamiento rápido y línea base fuerte. Para una MLP pura es más estable que embeddings aprendidos desde cero.

Arquitectura



5,125,396 parámetros entrenables.
La capa oculta concentra el 99.9 % (5.1 M).

Salida softmax + entropía cruzada categórica dispersa.
Dropout para controlar el sobreajuste.

Condiciones de entrenamiento

Optimizador	Adam ($\eta = 10^{-3}$)
Pérdida	Entropía cruzada
Batch	128
Épocas máximas	25
Parada temprana	`val_accuracy`, paciencia 5
Regularización	Dropout 0.3
Semilla	42 (determinista)
Hardware	CPU i5-9400F, 6 núcleos

Protocolo de comparación

9 configuraciones con la misma partición, semilla y criterio de parada:

- profundidad: 0, 1, 2 y 3 capas ocultas;
- dropout 0.3 vs. 0.5–0.4 y L2;
- vocabulario 20,000 vs. 10k;
- balanceo de clases;
- tasa de aprendizaje y L2.

Ajuste de hiperparámetros

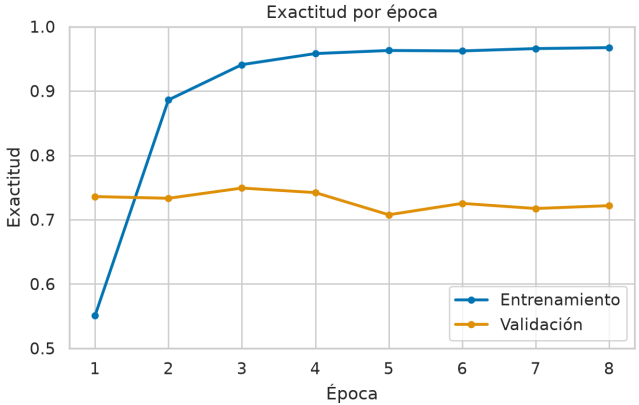
Configuración	Paráms.	Épocas	Exactitud	F1 macro
Lineal (sin capas ocultas)	0.4M	11	0.6572	0.6451
MLP 1×256 (principal)	5.1M	8	0.6771	0.6663
MLP 2×512–256	10.4M	10	0.6680	0.6578
MLP 2×512–256, dropout 0.5–0.4	10.4M	9	0.6644	0.6566
MLP 3×512–256–128	10.4M	13	0.6498	0.6433
MLP 2×512–256, vocab. 10k	5.3M	11	0.6439	0.6358
MLP 2×512–256, balanceado	10.4M	8	0.6686	0.6568
MLP 2×512–256, lr 5×10^{-4}	10.4M	10	0.6741	0.6653
MLP 2×512–256, L2 10^{-4}	10.4M	12	0.6455	0.6393

La primera capa oculta aporta la mayor ganancia; una tercera capa sobreajusta.

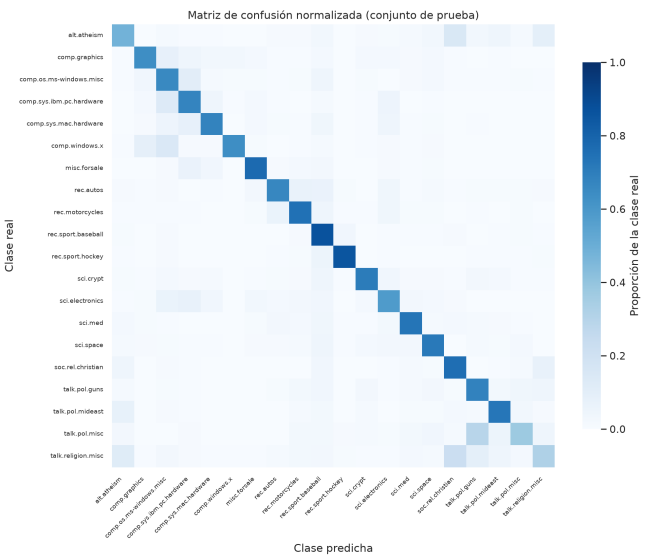
Resultados del modelo principal

Métrica	Valor
Exactitud	**0.6771**
Precisión macro	0.6786
Recall macro	0.6655
F1 macro	**0.6663**
F1 ponderado	0.6762
Exactitud top-3	0.8452

- Mejor época: 3 de 8.
- Validación: 0.7491.
- Brecha val–test: 7.20 p.p.



Matriz de confusión

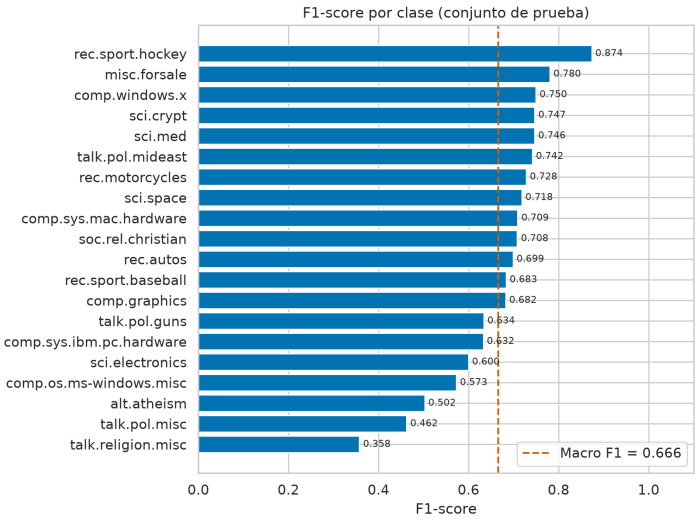


Errores estructurados:

- `comp.*`: hardware y sistemas operativos comparten vocabulario.
- `alt.atheism`, `soc.religion.christian` y `talk.religion.misc`: hilos de discusión mezclados.
- `talk.politics.*`: entidades y temas comunes.

Los documentos cortos y las respuestas citadas concentran los errores.

Desempeño por clase



Más fáciles

- `rec.sport.hockey`: 0.874
- `misc.forsale`: 0.780
- `comp.windows.x`: 0.750

Más difíciles

- `talk.religion.misc`: 0.358
- `talk.politics.misc`: 0.462
- `alt.atheism`: 0.502

El modelo lineal alcanza 0.6572; la MLP sube el F1 macro a 0.6663.

Conclusiones

- La MLP con TF-IDF alcanza **0.6771 de exactitud** y **0.6663 de F1 macro**, muy por encima del azar.
- Una capa oculta de 256 neuronas y dropout 0.3 ofrecen el mejor compromiso; añadir capas reduce el F1 macro y el vocabulario reducido pierde señal.
- El balanceo de clases no compensa la pérdida de exactitud.
- Los errores se concentran en clases semánticamente cercanas: el modelo no distingue el tema del documento de las palabras citadas.
- Limitación: *bag-of-words* ignora orden y contexto.

Siguiente entrega

Comparar una RNN sin memoria con una LSTM/GRU o Transformer, capaces de modelar secuencia y dependencias de largo plazo.

Demo ejecutable

Entrenamiento y evaluación

```
uv venv --python 3.12 .venv
uv pip install -r requirements.txt
.venv/bin/python -m src.experiments
```

Predicción interactiva

```
.venv/bin/python demo.py
.venv/bin/python -m src.predict "texto"
```

Salida esperada

```
Predicción: comp.windows.x (0.41)
-- comp.graphics (0.20)
-- comp.os.ms-windows.misc (0.14)
```

El notebook `entrega1_mlp.ipynb` reproduce el análisis y las figuras. Los artefactos quedan en `results/`.

Bibliografía


K. Lang. NewsWeeder: Learning to Filter Netnews. *ICML*, 1995.


T. Joachims. Text Categorization with Support Vector Machines. *ECML*, 1998.


D. P. Kingma, J. Ba. Adam: A Method for Stochastic Optimization. *ICLR*, 2015.


N. Srivastava et al. Dropout: A Simple Way to Prevent Neural Networks from Overfitting. *JMLR*, 2014.


F. Pedregosa et al. Scikit-learn: Machine Learning in Python. *JMLR*, 2011.